

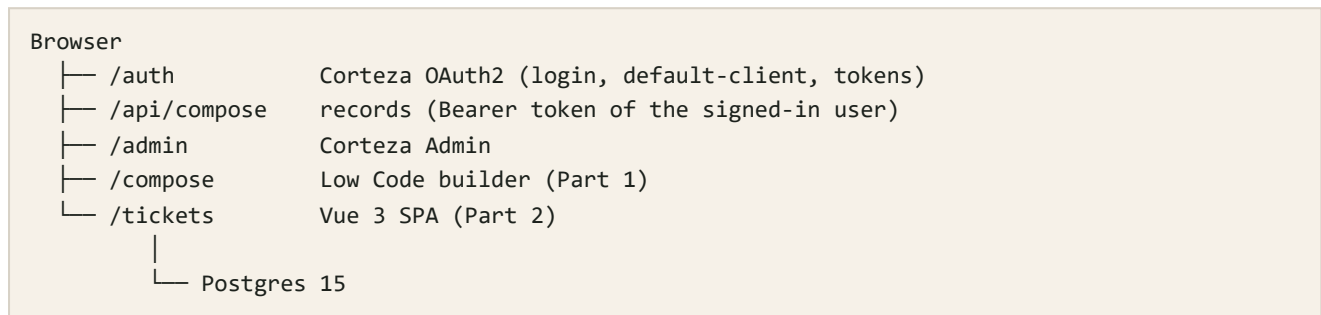
Support desk on Corteza

Vasil Filkin · Corteza 2024.9.9 · Vue 3 Composition API

Repo: <https://github.com/filkin1912/plexys-homework>

1. Architecture

One Corteza 2024.9.9 process serves Auth, the Compose API, Admin, Compose, and this SPA. PostgreSQL 15 stores system data and records. The desk is a first-party webapp (uses same host: localhost:18080) at /tickets, same origin as /auth and /api.



The SPA sends the user's access token. Corteza stamps ownedBy, createdBy, createdAt and updatedAt. Those fields are never written from the client.

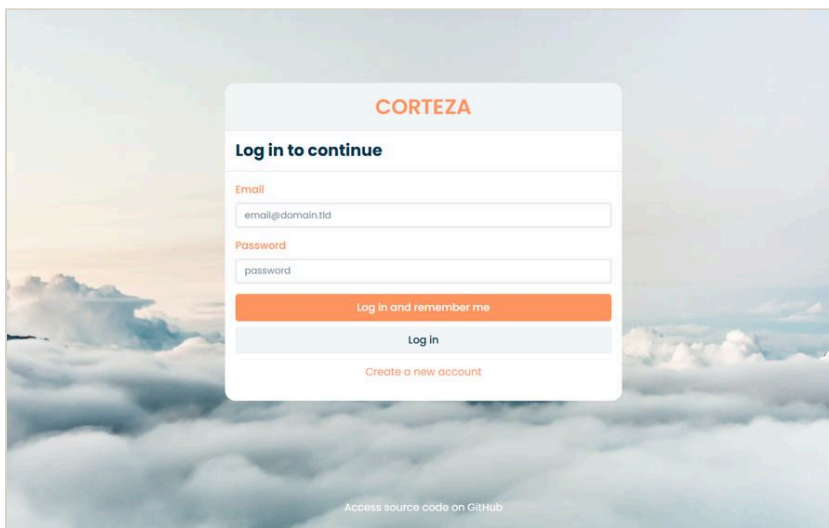


Figure 1. Local login. The desk reuses this session through the default OAuth2 client.

2. Decisions

Version. I ran Corteza 2024.9.9 — the latest stable Docker image that matches the current docs. The exact patch is pinned so a clone always starts the same version.

Serving. Corteza can serve a custom app next to Admin and Compose. I put the built desk in /corteza/webapp/tickets so it uses the same host and the same login as Compose. A separate Vite site on another port would need its own Auth Client and CORS. I did not build the desk as Compose pages — the brief asks for a custom SPA.

Vue 3 vs corteza-vue. The brief asks for Vue 3. The official corteza-vue package still uses Vue 2, so I did not use it. Ticket records go through corteza-js. Login is a small Vue 3 copy of Corteza's default-client flow (token in memory, refresh in sessionStorage).

UI. PrimeVue for the create/edit and delete dialogs. Tickets are a table so columns are easy to compare. Every action is a visible button.

3. Data model

Namespace **Plexys Homework** (plexys_homework). System owner and timestamp fields were not re-created. Customer is the bonus related module.

Module	Field	Type	Required
Support Ticket	Subject	String	Yes
Support Ticket	Description	String, multi-line	No
Support Ticket	Status	Select: New, In Progress, Resolved, Closed	Yes
Support Ticket	Priority	Select: Low, Medium, High, Urgent	Yes
Support Ticket	Due Date	DateTime	No
Support Ticket	Customer	Record → Customer	No
Customer	Name	String	Yes
Customer	Company	String	No
Customer	Email	Email	No

The screenshot shows a web application titled "Support desk" with the subtitle "PLEXYS HOMEWORK". It features a navigation bar with "Tickets" and "Customers" links. The user is logged in as "homework@plexys.local" and can click "Sign out".

Below the navigation bar, there's a section for "15-20 of 20 records". A search bar labeled "Filter by subject or customer" is present, along with a "All priorities" dropdown menu and a "New ticket" button.

The main content is a table with the following columns: SUBJECT, STATUS, PRIORITY, CUSTOMER, DUE, UPDATED, and ACTIONS. The table contains six rows of ticket data:

SUBJECT	STATUS	PRIORITY	CUSTOMER	DUE	UPDATED	ACTIONS
Extra login for night shift	Closed	Low	Nordic Freight	—	—	Edit Delete
Weekly report missing Saturday rows	Resolved	Low	Nordic Freight	—	—	Edit Delete
RFID gate stuck open at terminal 3	New	Urgent	Baltic Ports	8/23/2026, 9:00:00 PM	—	Edit Delete
Invoice portal returns 500 on export	In Progress	Medium	Nordic Freight	8/26/2026, 12:00:00 PM	—	Edit Delete
say Hello to Vasil	New	Medium	—	—	—	Edit Delete
say Hello	In Progress	Medium	Oslo Coldstore	9/1/2026, 12:21:00 PM	—	Edit Delete

At the bottom of the table, there are pagination controls: "Previous", "1", "2", "3" (selected), and "Next".

Figure 2. Tickets: list, search, priority filter, create / edit / delete.

Support desk PLEXYS HOMEWORK			Tickets Customers	Logged-in user: homework@plexys.local Sign out
Customers			New customer	
NAME	COMPANY	EMAIL	ACTIONS	
Oslo Coldstore	Oslo Coldstore AS	night@oslo-coldstore.example	Edit	Delete
Baltic Ports	Baltic Ports Oy	yard@baltic-ports.example	Edit	Delete
Nordic Freight	Nordic Freight AB	ops@nordic-freight.example	Edit	Delete

Figure 3. Customers: related-module CRUD used by the ticket picker.

4. User guide

1. Open `http://localhost:18080/tickets` and sign in as `homework@plexys.local` / Homework!2026.
2. The header shows the signed-in email. New tickets are owned by that user.
3. **New ticket** / **Edit** open a modal. Subject, status and priority are required. Pick a customer if one exists.
4. **Delete** asks for confirmation.
5. **Customers** manages the related records. Create a customer before linking it on a ticket.
6. Sign out returns to Corteza Auth.

5. Admin guide

How to start the stack is in the README. After `docker compose up`, wait until `http://localhost:18080/version` shows 2024.9.9.

The first start creates the demo user, the modules, and about 20 tickets. If login fails or the list is empty, run `docker compose down -v` in `corteza/` and start again.

Auth. No extra Auth Client. The desk uses Corteza's built-in login, same as Compose.

Config. Settings are in `corteza/.env.example`. After a UI change, run `npm run build` in `spa/` and refresh `/tickets`. Modules can also be checked in Compose at `/compose`.

6. Limitations and next

- The list loads at most 200 tickets. Next: proper server-side paging.
- Anyone who can sign in can use the desk. Next: a Support Agent role.
- Deleting a customer does not update old tickets that still point to it.
- No automated browser tests yet.

Observed: signup works without email; `/version` is 2024.9.9; Corteza fills owner and timestamps on create. Assumed: `/tickets/auth/callback` is served by the SPA, like other Corteza apps.